

Sustainable Kubernetes Extension Stack

Final Year Project

George Lipceanu (20103125)

October 14, 2025

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Review of Related Work	2
2	System Architecture and Design	2
2.1	CRDs	2
2.2	Controller Logic	3
3	Scheduler Design and Implementation	5
3.1	Implementation Structure	5
3.2	Filtering Phase	6
3.3	Scoring Phase and Scheduling Formula	6
3.4	Configuration and Extensibility	6
3.5	Modified Scheduler Design for Sustainability	6
3.5.1	Design Objectives	6
3.5.2	Implementation Details	7
3.5.3	Configuration Example	7
3.5.4	Emission Estimation	7
3.6	Impact and Discussion	8
4	Implementation of Supporting Components	8
4.1	Kepler	8
4.2	Prometheus	9
4.3	Grafana	9

4.4	Cobra CLI	9
5	Multi-Cluster Deployments	9
6	Testing and Evaluation	9
7	Summary and Discussion	9

1 Introduction

This project delivers an opinionated, end-to-end sustainability extension stack for Kubernetes. It integrates *Custom Resource Definitions (CRDs)*, *reconciling controllers*, and a *carbon-aware scheduler plugin* into a cohesive system that makes energy and carbon first-class signals in cluster operations. A companion CLI automates installation, upgrades, and cluster validation, while an observability layer (Prometheus + Grafana) exposes real-time power, energy, and emission metrics.

Problem Statement Kubernetes optimises for resource fit, availability, and fairness, but lacks native constructs to reason about energy or carbon. Operators cannot declaratively express sustainability intent, like favouring lower-intensity nodes. or audit the energy and emissions impact of workloads without ad-hoc tooling.

Vision Make sustainability declarative, actionable, and measurable for Kubernetes workloads by: (i) modelling carbon and energy as API resources; (ii) reconciling cluster state toward sustainability intent; and (iii) feeding those signals into scheduling and reporting decisions.

Scope The stack provides:

- **APIs:** CRDs for carbon intensity, workload policies, and emission reports.
- **Controllers:** Operators that fetch carbon data, apply workload policies, and provide data on energy saved, emissions, etc..
- **Scheduler Plugin:** Scheduler that combines standard utilisation bin-packing with carbon-aware scoring.
- **External Components:** Kepler for node-level power estimation, Prometheus for metric collection, Grafana for dashboards.
- **Tooling:** CLI binaries to install, configure, and inspect the stack and clusters data.

Design Principles

1. **Upstream compatibility:** built on the Kubernetes Scheduling Framework and CRD APIs.
2. **Separation of concerns:** making sure each component in the system does one thing well.
3. **Clear intent:** sustainability is defined through CRDs and labels.
4. **Observability by default:** all sustainability data are inspectable via metrics and dashboards.
5. **Fault tolerance:** provider failures fall back to last-known-good or neutral scores.

Implementation Overview The system is implemented in Go using Kubebuilder for operator scaffolding. The custom scheduler plugin ingests (i) node utilisation, (ii) carbon intensity signals from CRDs or cached ConfigMaps, and (iii) workload policies to compute a composite score per node. Prometheus scrapes Kepler and controller metrics, with Grafana visualising energy, emission, and placement data. The CLI automates installation, upgrades, and basic diagnostics.

1.1 Motivation

Cloud workloads are increasingly energy-intensive, yet most platforms obscure their carbon impact. Teams cannot easily answer:

1. Where did this workload run?
2. How much energy did it consume?
3. Could this energy use have been reduced?

With AI/ML and data-processing clusters growing rapidly, a practical way to measure, optimise, and report sustainability at the orchestration layer is timely. This project addresses that gap through three core capabilities:

1. **Measurement:** Kepler-based power and energy estimation linked to workloads over time.
2. **Optimisation:** a scheduler that prefers lower-intensity, highly-utilised nodes (bin-pack + carbon scoring).
3. **Reporting:** CRD-driven policies and auditable emission reports for teams and compliance, alongside observability in Grafana dashboards.

1.2 Review of Related Work

Energy/Carbon-Aware Scheduling Existing research and community prototypes extend scheduling logic with sustainability signals, often combining resource fit with energy or carbon metrics. This work follows that approach but maintains upstream compatibility through the Kubernetes Scheduling Framework, avoiding invasive patches to the core scheduler.

Telemetry and Power Modelling Kepler estimates node and container energy using eBPF (extended Berkeley Packet Filter) and hardware counters, exporting metrics consumable by Prometheus. For real-time scheduling, a linear node-level power model is used as shown below:

$$P = P_{\text{idle}} + U_{\text{cpu}} (P_{\text{max}} - P_{\text{idle}})$$

Policy and Governance This project expresses sustainability constraints declaratively through CRDs such as `WorkloadPolicy`, allowing schedulers and controllers to consume consistent signals without bespoke integrations for each workload.

Carbon Data Providers Operational systems commonly query regional carbon intensity from external providers. This stack encapsulates provider interactions in the `CarbonInfoController`, which normalises units, updates CRD status, and writes a cached ConfigMap. The scheduler reads from this cache, ensuring deterministic behaviour even under provider outages.

Positioning This work is fully Kubernetes-native. Sustainability intent is declared via CRDs, enforced by reconcilers, enacted at scheduling time, and validated post-hoc through emission reports and metrics. The emphasis is on practicality—achieving measurable sustainability impact with minimal operational overhead.

2 System Architecture and Design

2.1 CRDs

Custom Resource Definitions capture sustainability signals and preferences as Kubernetes-native APIs. They include OpenAPI validation, defaults, and standard status conditions and are built using Kube-builder. Controllers reconcile these resources and project their outputs to the scheduler and observability stack.

Summary of CRDs

- **CarbonInfo:** normalises grid carbon intensity per region or node group.
- **WorkloadPolicy:** defines carbon-aware scheduling preferences and thresholds for labelled workloads.
- **EmissionReport:** aggregates energy and carbon emissions over a reporting window.

CarbonInfo *Purpose:* represent regional or node-group carbon intensity as a live signal.
Spec (key fields)

- **provider:** data source (mock, ElectricityMap, or static).
- **region:** provider-specific key.
- **pollSeconds:** refresh cadence for live data.
- **staticIntensity:** fallback or fixed gCO₂/kWh for static mode.

Status (key fields)

- **currentIntensity:** latest gCO₂/kWh value.
- **lastUpdated:** timestamp of last successful update.
- **conditions:** readiness and error states.

WorkloadPolicy *Purpose:* declaratively express carbon-aware scheduling preferences for matching workloads.

Spec

- **target:** label and namespace selectors defining workloads in scope.
- **maxCarbonIntensity:** upper bound (gCO₂/kWh) for eligible nodes.
- **schedulerHints:** optional key-value hints such as `utilisationWeight: 0.7`, `carbonWeight: 0.3`.
- **enforcement:** `soft` (prefer) or `hard` (require).

Status

- **enforced:** whether hints are applied to all matching workloads.
- **matchedWorkloads:** count of affected workloads.
- **conditions:** reconciliation results.

EmissionReport *Purpose:* summarise cluster-wide or workload-level energy and emission metrics into auditable reports.

Spec

- **scope:** aggregation level (namespace, workload, or node).
- **from, to:** timestamps defining the reporting window.

Status

- **totals:** overall energy (kWh) and emissions (gCO₂) for the report period.
- **breakdown:** per-namespace/workload/node details.
- **conditions:** `Finished` or `DataPartial` with current info.

Validation and Security CRDs include OpenAPI validation to enforce schema correctness. Controllers run with minimal RBAC privileges and operate namespace-scoped for multi-tenant isolation.

2.2 Controller Logic

Controllers follow the operator reconciliation pattern to align cluster state with declarative sustainability intent. Each controller observes its CRD, updates **status**, and publishes derived data to the scheduler or metrics system.

CarbonInfoController

- **Watches:** CarbonInfo resources.
- **External I/O:** queries carbon intensity from configured providers.
- **Actions:** updates CRD status, maintains `ready/unavailable` conditions, and writes a cached ConfigMap mapping nodes or regions to intensity values for the scheduler plugin.

WorkloadPolicyController

- **Watches:** WorkloadPolicy and corresponding Jobs/Pods/Deployments.
- **Actions:** validates policy and injects hints via annotations and labels, sets `spec.schedulerName: carbon-scheduler` when required, updates `status.enforced`.
- **Scheduler Integration:** the scheduler plugin reads these hints during the scoring process to adjust weighting and eligibility.

EmissionReportController

- **Watches:** EmissionReport resources.
- **Inputs:** Prometheus and Kepler metrics (CPU usage, power), CarbonInfo intensities.
- **Actions:** queries metrics over the report window, calculates energy consumption using the linear power model, multiplies by carbon intensity to estimate emissions, and writes totals and breakdowns to `status`.

Data Flow Summary

1. **CarbonInfoController** fetches carbon intensity and updates CRD status and cache.
2. **WorkloadPolicyController** injects scheduling thresholds and pre-reqs into targeted workloads.
3. **Scheduler Plugin** reads utilisation, policies, and cached carbon data to score nodes and bind pods.
4. **EmissionReportController** queries metrics and produces reports of energy and emissions.

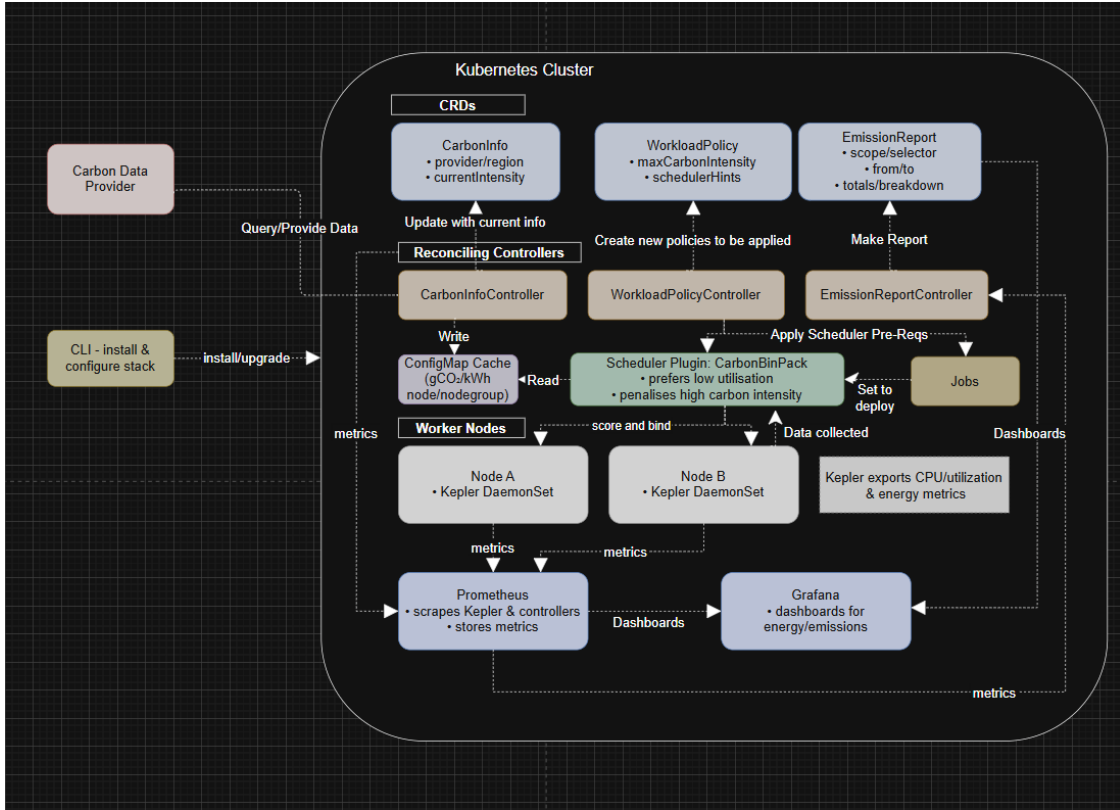


Figure 1: Overall architecture of the Sustainable Kubernetes Extension Stack.

Operational Notes

- Controllers are idempotent and reconcile periodically or on CRD change events.
- The controllers pre-compute data required for constant-time scoring for scheduler, minimising scheduling time.

3 Scheduler Design and Implementation

The Kubernetes scheduler is responsible for assigning unscheduled Pods to available Nodes. When a new Pod is created without a `spec.nodeName`, the scheduler sees it, evaluates all available Nodes, selects the best Node according to cluster policy, and writes the Node name back to the Pod object. The scheduler continuously watches for pending Pods and reacts to changes in cluster state and its primary goal is to ensure efficient resource utilisation and fairness across workloads.[\[1\]](#)

3.1 Implementation Structure

The scheduler source code can be found in the `pkg/scheduler/` directory of the Kubernetes repository. The main interfaces are:

- **Filter** — decide whether a Node is feasible.
- **Score** — rank feasible Nodes with an integer 0–100.
- **PreBind / Bind / PostBind** — finalise scheduling.

Each phase is implemented through the *Scheduling Framework*, which provides plugin interfaces located in `pkg/scheduler/framework/`. This modularity allows custom policies to be added without modifying core logic [\[2\]](#).

3.2 Filtering Phase

In the filtering phase, the scheduler evaluates each Node against a set of constraints implemented by **Filter plugins**. Typical filters include:

- **NodeResourcesFit**: Ensures that the Node has sufficient CPU and memory.
- **TaintToleration**: Verifies that the Pod can tolerate Node taints.

Nodes that pass all filters are added to the list of feasible Nodes.

3.3 Scoring Phase and Scheduling Formula

Once feasible Nodes are determined, each is scored by multiple **Score plugins**. These plugins assign integer scores in the range [0, 100]. The final scheduling decision is based on a weighted combination of all plugin scores.

3.4 Configuration and Extensibility

The scheduler is configured using a `KubeSchedulerConfiguration` object. An example configuration is shown below:

```
apiVersion: kubescheduler.config.k8s.io/v1beta3
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: default-scheduler
  plugins:
    score:
      enabled:
        - name: NodeResourcesFit
  pluginConfig:
    - name: NodeResourcesFit
      args:
        scoringStrategy:
          type: MostAllocated
```

Pods can opt into a specific scheduler profile by setting:

```
spec:
  schedulerName: default-scheduler
```

This configuration-based design allows different workloads to use different scheduling policies within the same cluster.

3.5 Modified Scheduler Design for Sustainability

While the vanilla Kubernetes scheduler focuses on resource allocation and fairness, it does not consider the environmental impact of workload placement. This project extends this mechanism by introducing carbon-aware scheduling, enabling Pods to be assigned to Nodes that minimise energy consumption and emissions. This is achieved through the addition of a new scoring plugin, implemented using the Kubernetes Scheduling Framework API. The plugin integrates data from sustainability metrics and resource utilisation to make more environmentally efficient scheduling decisions.

3.5.1 Design Objectives

The primary goals of the modified scheduler are:

- Reduce energy usage by consolidating workloads onto fewer, more efficiently utilised Nodes.
- Maintain compatibility with upstream Kubernetes.

3.5.2 Implementation Details

The plugin is implemented under `pkg/scheduler/carbonfit/plugin.go` and registered in `cmd/kube-scheduler/main.go`. The Scoring logic for the sample is shown below (based on other plug-ins):

```
func (p *Plugin) Score(ctx context.Context, state *framework.CycleState,
    pod *v1.Pod, nodeName string) (int64, *framework.Status) {

    ni, _ := p.handle.SnapshotSharedLister().NodeInfos().Get(nodeName)
    used := ni.Requested.MilliCPU
    alloc := ni.Allocatable.MilliCPU
    utilisation := float64(used) / float64(alloc)

    // Base score favors high utilisation (bin-packing)
    score := int64(utilisation * 100)

    // Incorporate carbon intensity from CRD
    carbonIntensity := 250.0 // gCO2/kWh (static example)
    penalty := int64((1 - utilisation) * carbonIntensity / 10)

    final := score - penalty
    if final < 0 { final = 0 }

    return final, framework.NewStatus(framework.Success, "")
}
```

This logic favors Nodes that are well utilised and penalises those with higher carbon intensity, balancing performance with environmental cost.

3.5.3 Configuration Example

```
apiVersion: kubescheduler.config.k8s.io/v1beta3
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: susk8s-scheduler
  plugins:
    score:
      enabled:
        - name: CarbonBinPack
        - name: NodeResourcesFit
  pluginConfig:
    - name: CarbonBinPack
      args:
        UtilisationWeight: 0.7
        CarbonWeight: 0.3
```

3.5.4 Emission Estimation

The total energy consumption of a Node over time is modeled as a function of its CPU utilisation. Each Node has a characteristic power profile defined by its **idle power** (P_{idle}) and **maximum power** (P_{max}). The scheduler assumes that power consumption grows linearly with CPU usage, following the linear CPU power model established by Lin et al. (2021), introduced in Section 2.2 of [3]. This model provides a balance between accuracy and computational simplicity, making it well suited for real-time scheduling decisions within a Kubernetes control loop. This makes the model sufficiently accurate for cluster-level energy and carbon estimations while remaining architecture-agnostic and lightweight to compute.

The instantaneous power draw E_n (in watts) for a Node n is thus estimated as:

$$E_n = P_{idle} + U_{cpu} \times (P_{max} - P_{idle})$$

where:

- E_n is the estimated node power draw in watts.
- P_{idle} represents the baseline power consumed when the node is idle.
- P_{max} represents the power at full CPU utilisation.
- U_{cpu} is the current CPU utilisation ratio, obtained from metrics exported by Kepler (`container_cpu_usage_seconds_total`).

This power is then integrated over the task’s duration, resulting in energy in joules or kilowatt-hours (kWh):

$$E_{total} = \int_{t_0}^{t_1} E_n(t) dt$$

Once the total energy consumed (E_{total}) is known, it can be converted into greenhouse gas emissions using the regional carbon intensity, I_{carbon} , measured in grams of CO₂ per kilowatt-hour (gCO₂/kWh). The emissions are calculated as:

$$CO_{2total} = E_{total} \times I_{carbon}$$

where:

- E_{total} is the total energy consumed (kWh).
- I_{carbon} is obtained from an external provider.

3.6 Impact and Discussion

This modification introduces sustainability as a first-class concern in Kubernetes scheduling. It reduces unnecessary energy usage and enables emission tracking. It also maintains full compatibility with upstream Kubernetes, demonstrating that sustainability can be achieved through simple plugin extensions and also allowing for compatible this same cluster deployment across multiple environments.

4 Implementation of Supporting Components

4.1 Kepler

Kepler (Kubernetes Efficient Power Level Exporter) is a lightweight component that estimates energy consumption and power draw at the container and node levels. It is deployed as a `DaemonSet` across all worker nodes and uses kernel-level counters and hardware performance counters to determine CPU power.

In this project, Kepler serves two key roles:

1. It provides node-level CPU utilisation and power estimation metrics (`kepler_container_joules_total`, `kepler_container_cpu_utilization`) that the `EmissionReportController` consumes.
2. It supplies instantaneous CPU utilisation values (`container_cpu_usage_seconds_total`) to the scheduler via Prometheus queries, enabling energy-aware placement decisions.

Kepler is deployed under the `kepler` namespace with a `ClusterRole` allowing node and pod metric access. Metrics are exposed at `/metrics` and scraped periodically by Prometheus. These values are aggregated and transformed into energy (Joules) and emission equivalents by the `EmissionReportController`, enabling correlation between workload identity, energy consumption, and carbon footprint.

4.2 Prometheus

4.3 Grafana

4.4 Cobra CLI

5 Multi-Cluster Deployments

6 Testing and Evaluation

7 Summary and Discussion

References

- [1] Kubernetes Authors. Scheduler design overview. <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>, 2025. Accessed: 2025-10-15.
- [2] Kubernetes SIG Scheduling. Kubernetes enhancement proposal: Scheduling framework. <https://github.com/kubernetes/enhancements/tree/master/keps/sig-scheduling/624-scheduling-framework>, 2025. Accessed: 2025-10-15.
- [3] Weiwei Lin, Tianhao Yu, Chongzhi Gao, Fagui Liu, Tengyue Li, Simon Fong, and Yongxiang Wang. A hardware-aware cpu power measurement based on the power-exponent function model for cloud servers. *Information Sciences*, 547:1045–1065, 2021. Accessed: 2025-10-12. doi:10.1016/j.ins.2020.09.033.
- [4] Suzanne Rivoire, Manasi Shah, Juan Meza, Rodrigo Azevedo, Satish Singh, Cenk Isci, and Frank Belloso. A comparison of high-level full-system power models. <https://rivoire.cs.sonoma.edu/pubs/rivoire.2008.comparison.hotpower.pdf>, 2008. Accessed: 2025-10-14.
- [5] Sustainable Computing IO Project. Kepler: Kubernetes efficient power level exporter. <https://github.com/sustainable-computing-io/kepler>, 2025. Accessed: 2025-10-15.